Engineering Journal

WRO Autonomous Driving Challenge

Team Name: Potato Nadoes
Country: Canada
Competition Year: 2026
Team Members: Nathan Li and Andrew Deng

YouTube Video: [Link]

Abstract

Our team developed an autonomous vehicle designed to sense walls and obstacles, adjusting to them. Throughout the project, we followed an iterative engineering process: we identified problems, developed solutions, tested them, analyzed the results, and redesigned the system where necessary. Our final vehicle combines servo motor Ackerman steering, DC motor 2WD with differential gears, Pi camera, and raspberry pi 5. Our proudest engineering achievement was [specific achievement], because it required us to solve [specific technical challenge] through multiple rounds of testing and refinement.

1. The Challenge & Our Systems Strategy

1.1 Understanding the Challenge

Before building the vehicle, we broke the overall autonomous-driving problem into smaller engineering systems:

Mechanical system – chassis, wheels, steering, drivetrain, mounting

Power system – battery, voltage regulation, motor power and electronics

Sensing system – cameras, distance sensors, encoders, or other sensors

Computing system – onboard computer and microcontrollers

Software system – perception, decision-making, control, and safety

Testing system – repeatable experiments, measurements, and performance evaluation

This decomposition allowed us to develop and test individual subsystems before integrating the complete vehicle.

1.2 Constraints

During development, we identified several important constraints:

Constraint	Why it matters	Example engineering impact
Vehicle size: 300 × 200 × 300 mm max	Limits chassis and component placement	We had to make the sensor mounts compact.
Vehicle weight: 1.5 kg max	Affects acceleration, traction, and motor load	We used lighter materials to leave weight for electronics.
4 wheels required	Limits drivetrain architecture	We couldn't use a differential-drive layout.
One steering actuator	Limits steering design	We needed a mechanically synchronized steering system.
Maximum 2 driving motors	Limits available driving power	Motor selection and gearing became important.
No omnidirectional/ball wheels	Limits wheel selection	We needed conventional wheels with a suitable steering geometry.
Fully autonomous operation	No human intervention during the run	Software needed reliable recovery and decision-making.
No RF/Bluetooth/Wi-Fi during competition	Prevents wireless control/communication	Debugging and inter-controller communication had to be wired or disabled.
Sensors are unrestricted	Gives freedom but creates integration challenges	We had to balance sensor capability against weight, space, processing, and power.
Battery choice is unrestricted	Lots of design freedom	We still had to balance capacity, weight, voltage, and current capability.
Only wired communication between electromechanical components	Limits communication architecture	We had to carefully plan wiring and connectors.
Fixed vehicle size during the run	Prevents expanding mechanisms from simplifying parking	Our final dimensions had to work for both driving and parking.

2. Mechanical Iteration & Mobility

2.1 Chassis Evolution

Rather than designing the final chassis immediately, we developed several versions.

Version 1: Chassis bought from amazon

Problem: The chassis that we purchased off amazon was not very modification friendly, very difficult to tear apart and did not have differential gears or a usable steering system

Test Result: We were unable to put our servo and DC motors to use and designing 3D prints was very difficult.

Lesson: This taught us that we can't truly be sure of what we're purchasing online as it may seem usable before we actually receive it.

Change for Version 2: We switched to a simple chassis with only the required things such as differential gears and a steering system.

Figure 1 – Version 1 prototype

Version 2

Problem identified from Version 1: The chassis had nothing to hold down the wheels

Modification: We added a wheel holder over to keep the wheels down using a 3D print

Test Result: It worked for a while but then broke, and we had to create new ones.

Lesson: We learned to always have spare parts in case the 3D prints broke.

Change for Version 3: [Change]

Figure 2 – Version 2 prototype

Version 3 – Final Design

The final chassis incorporated [major improvements]. These changes were based on the results of previous physical tests rather than solely on CAD assumptions.

Figure 3 – Final chassis

2.2 Mechanical Testing

One significant problem was the steering system.

During testing, we observed that the robot tended to drift off to one side. We determined that the likely cause was that the gear holder wasn't even. We changed the software and repeated the experiment.

The software change fixed the problem temporarily but reduced our steering angle and made the robot slightly less consistent.

So, we designed a better wheel holder, capable of giving each of the wheels equal grip on the ground

This demonstrated that although some problems are temporarily fixable through software solutions, removing the root is always the best solution.

3. Power & Sensor

3.1 Power Architecture

Our electrical system consists of Power supply Expansion board for Raspberry Pi 5, Power supply Expansion board for Raspberry Pi 5, HS-5055MG 11.9g Metal Gear Digital Micro Servo, and Pi camera.

Power Budget

Component	Voltage	Typical Current	Peak Current	Typical Power	Peak Power
Raspberry Pi 5	5 V	~1.2 A	~2.5 A	~6 W	~12.5 W
Power Supply Expansion Board	5 V	~0.1 A	~0.2 A	~0.5 W	~1 W
HS-5055MG Servo	4.8–6 V	~0.2–0.5 A	~1–1.5 A	~1–3 W	~5–9 W
Pi Camera	5 V	~0.1–0.2 A	~0.25 A	~0.5–1 W	~1.25 W
DC Motor	5 V	~0.5 A	~1.5 A	~2.5 W	~7.5 W
TOTAL		~2.1–2.5 A	~5.45–5.95 A	~10.5–13 W	~27–31 W

This is why we needed at least a 5V 6A battery

3.2 Sensor Selection

We selected the Pi camera because it worked well with the raspberry pi 5 that we were using.

3.3 Sensor Placement

Sensor placement was determined through testing rather than convenience.

For example, the Pi camera was positioned close to the top because it was able to see a large part of the map. We discovered that positioning was originally slightly too high and slightly on one side.

Through testing, we found the perfect position for the Pi camera.

4. Software Logic & Problem Solving

4.1 Software Architecture

Our software is divided into several stages:

Perception -> Decision Making (raspberry pi) -> Commands to Slave Controller (Arduino Nano) -> Motor Control

This separation allowed us to modify individual parts of the software without completely rewriting the system.

4.2 State Machine

The vehicle operates using a state-based control system.

Figure 7 – State machine / flowchart

Example states include:

Searching → Lane Following → Obstacle Detection → Avoidance → Recovery → Lane Following

Each state has defined inputs, actions, and transition conditions.

4.3 Control Tuning

For steering control, we used PD control, calculating both the Proportional (P) and Derivative (D) Control.

We adjusted both kp and kd while measuring how sharp each turn was and how consistent the robot was with the sharpness.

5. What We Would Improve

Engineering did not end with the final prototype. If we had additional development time, we would improve:

1. Even more compact

We would make the robot even more compact, having a lower center of gravity so it could be more stable.

2. Attach 2D Lidar

We would add a space for a 2D Lidar because it could help us complete tasks such as the parallel parking.

3.

We would [change] because [reason].

The most important next improvement would be [improvement], because it would have the greatest effect on [performance/reliability/safety].

6. Engineering Lessons

The most important lessons from this project were:

Test before assuming.
CAD models and theoretical calculations helped us predict performance, but physical testing frequently revealed unexpected behavior.

Measure instead of guessing.
We improved our designs when we replaced subjective observations with measurements and repeatable tests.

Subsystems are interconnected.
A mechanical change could affect sensors, power; consumption, software behavior, or vehicle stability.

Failure is useful engineering data.
Our most valuable improvements often came from failures because they revealed weaknesses that were not obvious during initial design.

Iteration produced our final vehicle.
The final design was not the result of one perfect idea. It was the result of repeated cycles of designing, building, testing, analyzing, and improving.

Appendix – Evidence Index

The Engineering Journal explains the engineering process, while our GitHub repository contains the detailed technical assets.

GitHub Repository

GitHub Repository: NateTheGrape32/WRO-future-engineers: WRO 2026 Andrew and Nathan

Contains:

Source code

CAD files

Wiring diagrams

Technical drawings

Vehicle photographs

Team photographs

Build instructions

Detailed README

Additional technical documentation

Video

[YouTube Link]